

## 1) howManyFromEach(Reaction[][] matrix)

O que o método tem de fazer: devolver um `int[]` em que a posição `i` contém quantas vezes aparece o `i`-ésimo elemento do enum `Reaction` dentro da matriz.

A solução tem de funcionar mesmo que o enum tenha “mais/menos” elementos.

O enum `Reaction` tem vários valores (LIKE, DISLIKE, ...) e um `toString()` (mostra emojis)

```
static int[] howManyFromEach(Reaction[][] matrix) {
    Reaction[] vals = Reaction.values();
    int[] counts = new int[vals.length];

    for (int r = 0; r < matrix.length; r++) {
        Reaction[] row = matrix[r];
        for (int c = 0; c < row.length; c++) {
            Reaction react = row[c];

            if (react != null) {
                counts[react.ordinal()]++;
            }
        }
    }
    return counts;
}
```

## B) O que é ligação dinâmica

A ligação dinâmica em Java consiste no mecanismo pelo qual o método a executar é escolhido em tempo de execução, de acordo com o tipo real do objeto e não com o tipo da variável de referência.

Por exemplo, considerando uma variável do tipo `IFornecedor` que referencia um objeto `Produtor` ou `Distribuidor`, a chamada ao método `precoUnidade` executa a implementação correspondente à classe concreta do objeto, permitindo comportamento diferente para fornecedores distintos através de polimorfismo.

## 2) `Reactable` (interface), `ReactableContact` (classe), `Reaction` (enum), `MarketingStrategy` / `EmailAdapter` (interfaces)

### A) `WebEnterprise` (classe abstrata)

- `WebEnterprise` é uma empresa mas também é um contacto `reactable`
- então: `extends ReactableContact`
- tem uma lista de contactos: `List<ReactableContact>`

```
public abstract class WebEnterprise extends ReactableContact {

    private List<ReactableContact> contacts;

    public WebEnterprise(String socialName, String eMail) {
        super(socialName, eMail);
        this.contacts = new ArrayList<>();
    }

    public Iterable<ReactableContact> getContacts() {
        return new ArrayList<>(this.contacts); // cópia
    }

    public void addContact(ReactableContact lc) {
        this.contacts.add(lc);
    }

    // Template method
    public void marketingCampaign() {
        giveLikeToContacts();
        askForLikeFromContacts();
    }

    public abstract void askForLikeFromContacts();
    public abstract void giveLikeToContacts();
}
```

```

@Override
public String toString() {
    String s = super.toString();
    s += "Contact names:\n";
    for (ReactableContact c : contacts) {
        s += c.socialName() + "\n";
    }
    return s;
}
}

```

## B) Classe concreta **MyEnterprise**

- dá likes a **todos**
- pede likes a **todos**
- pedir likes = enviar email com **EmailAdapter**

```

public class MyEnterprise extends WebEnterprise {

    private EmailAdapter email;

    public MyEnterprise(String socialName, String eMail, EmailAdapter email) {
        super(socialName, eMail);
        this.email = email;
    }

    @Override
    protected void giveLikeToContacts() {
        for (ReactableContact c : getContacts()) {
            c.receiveReactionFrom(this.socialName(), Reaction.LIKE, new Date());
        }
    }

    @Override
    protected void askForLikeFromContacts() {
        for (ReactableContact c : getContacts()) {
            email.sendMessage(c.eMail(), "Give a Like to " + this.socialName());
        }
    }
}

```

## C) Classe concreta **ProfEnterprise**

**Requisito:** aqui as escolhas são delegadas numa **MarketingStrategy**. Além disso, o **toString()** deve incluir o tipo de estratégia.

```

public class ProfEnterprise extends WebEnterprise {

    private MarketingStrategy strategy;

    public ProfEnterprise(String socialName, String eMail, MarketingStrategy strategy) {
        super(socialName, eMail);
        this.strategy = strategy;
    }

    @Override
    protected void giveLikeToContacts() {
        strategy.giveLikes(this);
    }

    @Override
    protected void askForLikeFromContacts() {
        strategy.askForLikes(this);
    }

    @Override
    public String toString() {
        return super.toString() + strategy.toString() + System.lineSeparator();
    }
}

```

## D) MarketingStrategy + NeverStrategy

O interface no enunciado: `giveLikes(WebEnterprise emp)` e `askForLikes(WebEnterprise emp)`

A classe `NeverStrategy`:

- **giveLikes**: dá likes aos contactos que **nunca receberam LIKE** da empresa
- **askForLikes**: pede likes aos contactos que **nunca deram LIKE** a empresa (envia mail "Give a Like to <empresa>")
- precisa de um `EmailAdapter` no construtor (nota do enunciado)  
exame2023

Como testar "nunca recebeu LIKE da empresa"?

- No contacto `c`, ver `c.lastReceivedReactionFrom(emp.socialName(), Reaction.LIKE) == null`

Como testar "nunca deu LIKE à empresa"?

- Na empresa `emp`, ver `emp.lastReceivedReactionFrom(c.socialName(), Reaction.LIKE) == null`

```
public class NeverStrategy implements MarketingStrategy {

    private EmailAdapter email;

    public NeverStrategy(EmailAdapter email) {
        this.email = email;
    }

    @Override
    public void giveLikes(WebEnterprise emp) {
        for (ReactableContact c : emp.getContacts()) {
            if (c.lastReceivedReactionFrom(emp.socialName(), Reaction.LIKE) == null) {
                c.receiveReactionFrom(emp.socialName(), Reaction.LIKE, new Date());
            }
        }
    }

    @Override
    public void askForLikes(WebEnterprise emp) {
        for (ReactableContact c : emp.getContacts()) {
            if (emp.lastReceivedReactionFrom(c.socialName(), Reaction.LIKE) == null) {
                email.sendMessage(c.eMail(), "Give a Like to " + emp.socialName());
            }
        }
    }
}
```

## 1b) Implementar `imprimeResultados`

O método deve, para cada `ReactableContact` de uma coleção, imprimir **e-mail e nome**.

E tem de aceitar as duas chamadas:

- `imprimeResultados(listaTres)`; onde `listaTres` é `ArrayList<ReactableContact>`
- `imprimeResultados(listaQuatro)`; onde `listaQuatro` é `ArrayList<WebEnterprise>` (e `WebEnterprise` é um `ReactableContact`)

```
static void imprimeResultados(Iterable<? extends ReactableContact> col) {
    for (ReactableContact c : col) {
        System.out.println("Mail: " + c.eMail() + " Nome: " + c.socialName());
    }
}
```

## GRUPO III

-Linha 5: `Date data1 = new Date;`

Erro: `new Date` falta `()` (estás a referir o construtor, não a chamar).

Correção: `Date data1 = new Date();`

**-Linha 8:** `Reactable r2 = new WebEnterprise("R2", "R2email");`

Erro: `WebEnterprise` é (tipicamente no enunciado) abstrata → não pode ser instanciada.

Correção: `Reactable r2 = new ReactableContact("R2", "R2email");`

**-Linha 10:** `MyEnterprise me1 = new WebEnterprise("me1", "me1@mail");`

Erro 1: `WebEnterprise` abstrata (mesmo problema).

Erro 2: mesmo que não fosse abstrata, `new WebEnterprise(...)` não é um `MyEnterprise` (tipos incompatíveis).

Correção: `MyEnterprise me1 = new MyEnterprise("me1", "me1@mail", emAdapt);`

**-Linha 16:** `ArrayList<Reactable> listaDois = new ArrayList<ReactableContact>();`

Erro (genéricos): `ArrayList<ReactableContact>` não é subtipo de `ArrayList<Reactable>` (invariância).

Correção: `ArrayList<Reactable> listaDois = new ArrayList<>();`

**-Linha 20:** `listaTres.get(0).marketingCampaign();`

Erro: `listaTres` é `ArrayList<ReactableContact>`. O tipo `ReactableContact` não tem o método `marketingCampaign()` (isso é de `WebEnterprise`).

Correção (mais direta): `cast ((WebEnterprise) listaTres.get(0)).marketingCampaign();`

**Método correto:**

```
private static void imprimeResultados(Collection<? extends ReactableContact> col) {
    for (ReactableContact rc : col) {
        System.out.println("Mail: " + rc.eMail() + " Nome: " + rc.socialName());
    }
}
```

Porquê ? `extends ReactableContact`?

Porque assim o método aceita:

- `Collection<ReactableContact>`
- `Collection<WebEnterprise>`
- `Collection<MyEnterprise>`
- etc.